

LAB ASSIGNMENT 10

PROBLEM 1: Apply Bisecting K-Means Algorithm of all three given datasets and check their performance.

CODE:

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

def plot_clusters(X, labels, title):
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)
    plt.figure()
    plt.scatter(X_pca[:,0], X_pca[:,1], c=labels, cmap='viridis')
    plt.title(title)
    plt.show()

def bisecting_kmeans(X, k, max_iter=10):
    clusters = {0: X}
    cluster_labels = np.zeros(len(X))
    current_k = 1
    while current_k < k:
        sse = {}
        for cid, points in clusters.items():
            if len(points) > 1:
                km = KMeans(n_clusters=1, random_state=42,
n_init=10).fit(points)
                sse[cid] = km.inertia_
        split_id = max(sse, key=sse.get)
        points_to_split = clusters[split_id]
        best_labels = None
        best_inertia = np.inf
        for _ in range(max_iter):
            km = KMeans(n_clusters=2, random_state=42, n_init=10)
            labels = km.fit_predict(points_to_split)
```

```

        if km.inertia_ < best_inertia:
            best_inertia = km.inertia_
            best_labels = labels
    new_id = max(clusters.keys()) + 1
    clusters[split_id] = points_to_split[best_labels == 0]
    clusters[new_id] = points_to_split[best_labels == 1]
    current_k += 1
final_labels = np.zeros(len(X))
label_counter = 0
for cid, points in clusters.items():
    indices = []
    for i, x in enumerate(X):
        if any(np.array_equal(x, p) for p in points):
            indices.append(i)
    for idx in indices:
        final_labels[idx] = label_counter
    label_counter += 1
return final_labels.astype(int)
df_d = pd.read_csv('diabetes_dataset.csv')
df_s = pd.read_csv('Social_Network_Ads.csv')
df_t = pd.read_csv('titanic.csv')
df_d.fillna(df_d.mean(numeric_only=True), inplace=True)
df_s.fillna(df_s.mean(numeric_only=True), inplace=True)
df_t['Age'] = df_t['Age'].fillna(df_t['Age'].median())
df_t['Embarked'] =
df_t['Embarked'].fillna(df_t['Embarked'].mode()[0])
df_t = df_t.drop(columns=['Cabin'])
datasets = {
    "Diabetes": (df_d, 'Outcome'),
    "Social Network Ads": (df_s, 'Purchased'),
    "Titanic": (df_t, 'Survived')
}
for name, (df, target_col) in datasets.items():
    X = df.drop(columns=[target_col])
    X = X.select_dtypes(include=[np.number]).values
    X_scaled = StandardScaler().fit_transform(X)
    labels = bisecting_kmeans(X_scaled, k=4)
    print(f"\n{name}")
    print("Cluster Counts:\n", pd.Series(labels).value_counts())
    print("Silhouette Score:", silhouette_score(X_scaled, labels))
    plot_clusters(X_scaled, labels, f"{name} Bisecting K-Means")

```

OUTPUT:

```
PS D:\NITJ ACADEMICS\OPENLEARN> python -u "d:\NITJ ACADEMICS\OPENLEARN\LAB_2026\LAB11_temp.py"
```

Diabetes

Cluster Counts:

2 275

1 223

3 136

0 134

Name: count, dtype: int64

Silhouette Score: 0.15237871994030108

Social Network Ads

Cluster Counts:

2 133

3 120

0 83

1 64

Name: count, dtype: int64

Silhouette Score: 0.23789252803153915

Titanic

Cluster Counts:

0 541

3 218

2 99

1 33

Name: count, dtype: int64

Silhouette Score: 0.31439725270801894



